Predictive Saturation Management with Conditional Certificate Transfer

Anonymous submission

Abstract

When a robot controller asks for more torque than the actuators can deliver, clipping keeps the actuators legal but silently breaks the behavior it was designed for. This paper keeps the controller and adds a much slower layer: the nominal controller — PD, impedance, trained policy, neural, or conditioned — runs unchanged at its own high servo rate, while a low-rate manager checks whether its request stays inside a robot-specific, uncertainty-tightened feasible set, applies a minimum-cost correction that keeps the predicted request inside that uncertainty-tightened realizability set, and reports directional-authority loss. A theorem gives conditions under which a velocity certificate transfers to a robot: the manager's torque prediction must be accurate, that prediction widened by its error bound must fit the actuator margin, and one-step velocity error must fit the certificate margin. A minimal instance of the certified action set and the tightened predicted-torque condition are enforced directly in the QP by construction; the realization-accuracy conditions remain robot-specific assumptions checked only by sampled audit. A deterministic benchmark matrix spans five controller interfaces, three realization maps, eight stress scenarios, and targeted ablations. Full-horizon enforcement removes a predicted $3.587\ \mathrm{Nm}$ future torque violation a first-step check misses; a cross-realization audit keeps all defects below $0.0069\ \mathrm{m/s}$ against a $0.03\ \mathrm{m/s}$ budget. It prevents violations under slow saturation, directional collapse, and near-boundary braking; disturbance and severe mismatch instead exceed the audit conditions. Only a velocity-restricted certificate is instantiated, not universal safety, a workspace-wide proof, or real-time performance.

Index Terms— physical interaction, actuator saturation, predictive constraint management, control refinement.

I. Introduction

A controller computes a torque request τ^0 and the hardware applies

$$\tau^{\mathrm{app}} = \mathrm{clip}\left(\tau^0,\ \tau_{\min},\ \tau_{\max}\right),$$

that is, each joint's torque is pushed back to its nearest limit whenever the request exceeds it. The clip protects the actuator. It does not protect the behavior. Once the clip is active, the acceleration the robot produces is not the acceleration the controller asked for, so the robot is no longer running the PD, impedance, or learned law that was designed or trained. Nothing in the loop announces this. The controller keeps issuing requests, the actuators keep saturating, and the realized closed-loop dynamics silently drift away from the intended ones. In physical interaction this matters directly, because the requested acceleration is what sets apparent compliance and disturbance response.

Different controllers reach this state through different routes. A fixed-gain PD law gets there through a large tracking error. An impedance law $M_d\ddot{e} = -K_d e - D_d\dot{e} + F_h$ gets there through contact force, even when tracking error is small. A $\tanh$-squashed learned policy bounds its requested *acceleration*, but the torque that acceleration implies depends on the configuration, so a bound on the request is not a bound on the torque. An upstream language- or diffusion-conditioned module may propose a motion primitive with no representation of the executing robot's actuator limits at all. The route differs; the failure is the same.

Keep the nominal controller unchanged. Add a much slower layer that predicts saturation and outputs a correction.

> *The nominal controller runs unchanged at its own high servo rate. A much slower manager rolls it forward over a short horizon, predicts whether the torque it is about to request will leave the feasible set, and if so applies a minimum-cost correction to the requested acceleration that keeps the request realizable — before hard clipping is required.*

The manager does not choose the task and does not replace the controller. Because its job is to predict and correct rather than react, it also does not need to run at servo rate; how slow it can run in general depends on the operating region and model (a concrete rate pair is instantiated in Section VII). Its only job is to keep the request physically producible while staying as close as possible to what the controller asked for. A final high-rate projection stays in place for disturbances that arrive between manager updates; prediction and last-resort protection are separate responsibilities, and Section VII shows they can be needed at different moments.

A one-step check answers “is the command legal right now?” That is not the same question as “will the controller be able to stay legal?” Position and velocity carry forward, so keeping a *future* step legal generally requires bending the acceleration *before* that step arrives. A constraint applied only to the currently applied command therefore permits a trajectory that walks into an infeasible future. Fig. 1 isolates exactly this: constraining every predicted move eliminates the planned torque excess in the horizon-ramp case, while constraining only the first move leaves a $3.587\ \mathrm{Nm}$ future violation and $31.180\ \mathrm{mm}$ of workspace excess (Section VII.B). Anticipation is not a refinement of reactive limiting; it is a different constraint.

Feedback linearization to a double-integrator-like behavior model is classical [1], [2]. Reference governors, predictive safety filters, anti-windup control, and model-predictive constraint handling are established [3]–[7]. We claim none of them. The double integrator here is a convenient shared interface, not a contribution.

The question we do ask is this:

> *How can future configuration-dependent loss of actuator realizability be anticipated without replacing the nominal interaction controller, and under what conditions does the resulting correction preserve a behavior-level certificate?*

Anticipation is the primary problem; certificate transfer is the conditional, supporting result that tells us when the correction can be trusted. Answering the second half requires *preserving* the robot-specific geometry rather than hiding it: the behavior dynamics give a shared place to predict and certify, while the feasible set and the uncertainty bounds stay robot-specific. Section VI turns that split into three checkable conditions.

Contributions.

1. A two-rate architecture that retains an existing high-rate controller and uses a much slower MPC only to anticipate and correct loss of actuator realizability.
2. A robot-dependent feasible-acceleration set and a directional-authority indicator logged at every predictive update, exposing failures hidden by scalar torque utilization.
3. A conditional certificate-transfer theorem, stated as three checkable conditions, separating a reusable velocity-certified action set from robot-specific actuator and model-error tests, together with a minimal constructive instance of that set enforced directly inside the QP.
4. A reproducible study of 111 deterministic runs, including repeated anchor cases across matrices (parameters in Appendix A; code, saved metrics, tests, and figure-generation scripts in the supplementary archive), covering controller substitution, a sampled cross-realization interface audit, horizon-wide constraints, uncertainty tightening, the final high-rate projection, the enforced velocity certificate, and failure outside the tested operating region.

The experiments are reduced-order and intentionally include negative cases. They establish the mechanism and its logical limits, not hardware-level universality.

Notation. Everything in the paper uses the following symbols. All vector inequalities are componentwise (joint by joint).

Symbol	Meaning in words	Units
τ^0	torque the nominal controller asks for	Nm
τ^{pre}	torque that would be sent *before* any clipping	Nm
τ^{app}	torque actually applied, after clipping	Nm
$\hat{\tau}$	the manager's *prediction* of τ^{pre}	Nm
$\tau_{\min}, \tau_{\max}$	actuator limits	Nm
τ_{base}	torque already spoken for: gravity, Coriolis, orientation, null-space	Nm
F_h	measured interaction force	N
$G_F(x)$	converts F_h into task-acceleration units; general form $\Lambda^{-1}(x)$, this benchmark uses $\frac{1}{m}I$	1/kg
a	requested task acceleration (the decision variable)	m/s²
a^0	acceleration the nominal controller asks for	m/s²
$\Delta a = a - a^0$	the correction the manager applies	m/s²
p, v	task position and velocity, propagated directly	m, m/s
$z = [p; v]$	task state	—
$e_c = p - p_r(t)$	a controller's own tracking error against its reference $p_r(t)$; internal to the controller, not part of z	m
$\mathcal{X}$	running workspace-and-speed box on z	—
$H(x)$	maps a requested acceleration to the torque it costs	Nm·s²/m
$\mathcal{A}(x)$	accelerations that are interface-realizable — legal through $H(x)$ and $\tau_{\text{base}}(x)$ — at this configuration	m/s²
$\mathcal{A}^{\text{tight}}(x)$	same set, shrunk by the uncertainty margin	m/s²
$\bar{\delta}_\tau$	how far the real torque can differ from the prediction	Nm
α^+	remaining acceleration authority in a given direction	m/s²
$\mathcal{S}_v = \{y : \|y\|_\infty \le v_{\max}\}$	the certified velocity region	—

Symbol	Meaning in words	Units
ϵ_v	certificate margin: the one-step velocity error the certificate tolerates	m/s
$\mathcal{K}_v(y)$	the certified action set: requests from y that stay inside $\mathcal{S}_v$ after up to ϵ_v of error	m/s²

Two set operations appear in Section VI only, and both have a plain reading: $\mathcal{X} \oplus \mathcal{Y}$ means "any point of $\mathcal{X}$ plus any point of $\mathcal{Y}$" (worst case over both), and $\mathcal{X} \subseteq \mathcal{Y}$ means "$\mathcal{X}$ fits inside $\mathcal{Y}$."

II. How the Method Works

This section walks through the loop once, before any of it is formalized.

Fast loop, at the controller's own high servo rate T_f, regardless of which controller is in use. Evaluate the nominal controller to get its requested acceleration a^0, converted to torque τ^0 through the realization map (Section IV). Add the latest correction published by the manager, converted to torque at the *current* configuration:

$$\tau^{\text{pre}} = \underbrace{\tau^0}_{\text{controller}} + \underbrace{H(x)\,\Delta a}_{\text{manager's correction}} ,$$

then apply the final high-rate projection, the last-resort guard,

$$\tau^{\text{app}} = \text{clip}\left(\tau^{\text{pre}},\ \tau_{\min},\ \tau_{\max}\right),$$

which is what is sent to the robot.

Slow loop, every manager period Δt, much longer than T_f. The manager publishes a correction in *acceleration*, not a cached torque; $H(x)$ is recomputed on every fast-loop tick, so configuration drift between manager updates does not turn a stale plan into an artificial disturbance. Each update:

1. Roll the nominal controller forward $N = 12$ steps to get the requested accelerations $a_0^0, \ldots, a_{N-1}^0$ and the predicted states.
2. At each predicted state, build the set of accelerations the robot can actually realize, shrunk by the uncertainty margin.
3. Solve one small QP for the acceleration sequence closest to the nominal request that stays inside those sets, inside the workspace box, and inside the velocity-certificate set of Section VI.
4. Publish the first-step correction Δa; log the margin and directional authority.

If the nominal rollout is already feasible everywhere, the QP returns the nominal request and the correction is zero — the manager is inactive during ordinary operation.

Where the anticipation lives. There is no separate "saturation detector." Along the nominal rollout the predicted torque is $\hat{\tau} = \tau_{\text{base}} + H(a^0 - G_F \hat{F}_h)$; when that torque enters the tightened boundary layer at any step of the horizon, the corresponding constraint becomes active and the QP is forced to move. The tightening is what makes the constraint bite *early*; the forward propagation of position and velocity is what turns a future activation into a present correction.

III. Related Work

Impedance control specifies a desired dynamic relation between motion and interaction force [1]; operational-space control maps task accelerations or wrenches to joint torque [2]. Both presuppose sufficient actuator authority. Direct clipping preserves torque bounds but changes the realized dynamics. Anti-windup methods address saturation-induced performance and stability degradation directly [6], [7], but are typically designed around a particular closed-loop controller and react to saturation rather than forecasting a configuration-dependent loss of task authority.

Model-reference adaptive impedance control asks the robot to reproduce a prescribed impedance model despite uncertain dynamics [11]. This is close to our behavior-coordinate viewpoint but does not by itself provide finite-horizon actuator-constraint management. We

instead retain the nominal interaction controller and alter its requested acceleration before clipping is predicted; the final projection is kept as a last-resort guard.

Reference and command governors modify a reference supplied to a pre-stabilized system so predicted constraints remain satisfied [3]. Predictive safety filters modify nominal actions using a predictive model and a recoverable terminal condition [5]. Robot-specific MPC can incorporate full nonlinear dynamics and actuator constraints directly. These establish that predictive constraint management is valuable; we do not claim otherwise.

Recent interaction-control work combines MPC with impedance adaptation, optimizing impedance parameters or trajectories directly using learned interaction dynamics, Gaussian-process task models, or environment estimation [12]–[15]. Our manager instead treats the upstream controller as fixed and selects a physically realizable acceleration sequence close to its request.

This does not make the optimizer categorically distinct from a reference governor or predictive safety filter — either architecture could implement the manager. The contribution is the behavior–realization split and the sufficient transfer condition, not a new name for constrained MPC. Unlike predictive safety filters with a verified terminal safe set [16], the present implementation enforces finite-horizon state and actuator constraints but does not establish recursive feasibility.

Control barrier functions are a natural high-rate mechanism for state inequalities [4] and are used here by the reactive baseline and the manager's own infeasibility fallback (Section V.A), but an instantaneous constraint may act only after the state has reached a region where authority is already insufficient. Our predictive layer evaluates future feasibility over a horizon; the present implementation uses no terminal recoverability condition.

The theoretical lineage of Section VI is approximate simulation and control refinement [8], [9], where an interface maps an abstract input to a concrete one while bounding the resulting mismatch, and contract-based design [10], which separates a component's assumptions from its guarantees. We specialize that logic to actuator-limited robot realization: the mismatch is built from a tightened actuator set, a torque-prediction error bound, and a one-step model-error bound. That is what makes the transfer conditional and falsifiable rather than assumed.

IV. What "Realizable" Means for a Given Robot

For a torque-controlled robot,

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau + J(q)^\top F_h,$$

with $M(q) \succ 0$, h the modeled gravity/Coriolis/friction terms, and F_h the measured interaction force (a two-dimensional vector in this benchmark, not a full wrench). Actuator limits are the box $\tau_{\min} \le \tau \le \tau_{\max}$.

The architecture asks only three things of the nominal controller: its **current requested task acceleration** a^0; an optional **preview**, i.e. the ability to be evaluated along a predicted trajectory; and a **bound on how wrong that preview can be**. An analytic PD or impedance law can be evaluated along predicted states directly. A learned policy can be queried on predicted observations. If no validated preview exists, zero-order hold or a learned predictor may be used, but the resulting error must be included in the bound. A black-box controller with neither query access nor a bounded preview error lies outside the certificate. The induced torque τ^0 is obtained from a^0 through the realization map below.

With task state $z = [p; v]$ — absolute task position and velocity, propagated directly rather than relative to a possibly time-varying reference — the requested behavior over a local operating region is

$$\ddot{p} = a + d_{\text{res}},$$

in words: the acceleration the task actually realizes equals the requested acceleration plus a residual disturbance d_{res}. Once the interaction-force compensation of (IV.0) below is introduced, d_{res} excludes F_h's own (compensated) contribution and carries only what that compensation misses — residual force-model error, cross-coupling, discretization — checked explicitly as a one-step model error in Section VI. A reference-tracking controller may separately form its own tracking error $e_c = p - p_r(t)$ against a reference $p_r(t)$ to compute its request; this controller-internal error is not itself part of z.

Over one manager period this integrates to the constant-acceleration update

$$p^+ = p + \Delta t\, v + \tfrac{1}{2}(\Delta t)^2(a + d_{\text{res}}), \qquad v^+ = v + \Delta t\,(a + d_{\text{res}}),$$

which is the standard $z^+ = Az + B(a + d_{\text{res}})$ double integrator. It is a shared interface, not a claimed novelty.

Realizing a requested acceleration a costs torque. Some torque is already spoken for — gravity, Coriolis, the orientation channel, null-space damping — and we collect all of it in $\tau_{\text{base}}(x)$. The realization map also compensates the measured interaction force F_h as a feedforward term, pricing a net of the force's own contribution: subtracting $G_F(x)F_h$ from the request before converting the rest to torque cancels F_h's own contribution to the realized acceleration exactly, since $H(x)G_F(x)F_h = J(q)^\top F_h$ matches the force's own term in the governing dynamics above — so a is the desired *total* task acceleration, not a residual defined against a separate disturbance term. This is independent of whether a controller's own request already reasons about F_h (as the impedance law of Section I does); the map applies the same compensation underneath every interface. What remains is proportional to the corrected request:

$$\tau = \tau_{\text{base}}(x) + H(x)\left(a - G_F(x)F_h\right), \qquad H(x) = J(q)^\top \Lambda(q), \quad \Lambda = \left(JM^{-1}J^\top\right)^{-1}, \quad G_F(x) = \Lambda^{-1}(x). \tag{IV.0}$$

The reduced-order benchmark of Section VII instead uses the scalar approximation $G_F = \frac{1}{m} I$ (an effective task-space mass m, Appendix A), exact only when $\Lambda \approx mI$.

We assume $J(q)$ has full row rank throughout the verified operating region and that $JM^{-1}J^\top$ is uniformly nonsingular there, $\lambda_{\min}(JM^{-1}J^\top) \geq \underline{\lambda} > 0$; configurations violating this condition lie outside the certificate. A damped operational-space inverse may be used in implementation near such configurations, but its acceleration-realization defect must then be folded into $\bar{\delta}_\tau$ (torque domain) or the one-step model-error bound (velocity domain) rather than assumed away.

In words: $H(x)$ is the price, in joint torque, of one unit of net task acceleration at this configuration, after the force compensation of (IV.0). A request is realizable only if that price fits in the remaining budget:

$$\tau_{\min} \;\leq\; \tau_{\text{base}}(x) + H(x)\left(a - G_F(x)F_h\right) \;\leq\; \tau_{\max}. \tag{IV.1}$$

Call the set of a satisfying (IV.1) the **feasible request set** $\mathcal{A}(x)$. This is the object that cannot be made robot-independent: it moves with the configuration through both $H(x)$ and $\tau_{\text{base}}(x)$, and removing that dependence would remove exactly the geometry that causes saturation.

Interface consistency. The nominal interface supplies a requested task acceleration a^0, either directly — as every controller evaluated in Section VII does — or, for an opaque torque-only controller, through a robot-specific conversion satisfying the residual bound below. The induced nominal torque is

$$\tau^0 = \tau_{\text{base}}(x) + H(x)(a^0 - G_F(x)F_h) + r_\tau(x), \qquad |r_\tau(x)| \leq \bar{r}_\tau,$$

where r_τ is an interface-consistency residual, bounded by the tightening margin $\bar{\delta}_\tau$ below; the five interfaces evaluated in Section VII are constructed so $r_\tau \equiv 0$.

Condition (IV.1) is a set of parallel slabs in a, one pair per joint, so $\mathcal{A}(x)$ is already a polytope in acceleration space.

Two practical consequences follow, and only these are used later.

- **It is not a ball.** Authority is direction-dependent. The robot can have plenty of torque left overall and still be nearly unable to accelerate along one particular direction.
- **It is cheap to probe.** Directional authority is evaluated directly from the half-space representation of $\mathcal{A}^{\text{tight}}(x)$, as defined in Section V.B; no vertex enumeration is required.

The manager predicts $\hat{\tau}$; the torque that would actually be sent, τ^{pre}, differs from it. Suppose that difference is bounded joint by joint:

$$|\tau^{\text{pre}} - \hat{\tau}| \;\leq\; \bar{\delta}_\tau. \tag{IV.2}$$

Then it is not enough for the *prediction* to sit inside the box; the prediction plus its error must. So the manager enforces the **tightened** condition

$$\tau_{\min} + \bar{\delta}_\tau \;\leq\; \hat{\tau} \;\leq\; \tau_{\max} - \bar{\delta}_\tau, \tag{IV.3}$$

and we write $\mathcal{A}^{\text{tight}}(x)$ for the accelerations satisfying it. Equation (IV.3) is the whole tightening mechanism: a boundary layer of width $\bar{\delta}_\tau$ inside each actuator limit. It is what makes the constraint act *before* the physical limit rather than at it, and Section VII.E shows what happens when it is removed.

The bound $\bar{\delta}_\tau$ must cover state-estimation error, interpolation between manager updates, secondary torque, torque-rate limiting (if present — this implementation has none), and any other implementation effect that can move the pre-clip torque. The implementation uses two concrete instantiations of it: a conservative, action-independent $\bar{\delta}_\tau^{\mathrm{QP}}$, evaluated at the acceleration limit rather than the not-yet-chosen request, inside the QP's own tightening; and a smaller, action-dependent $\bar{\delta}_\tau(a, F_h)$, evaluated at the request actually solved for, used in the (T1) audit of Section VII. Since the QP's bound is evaluated at a worst-case acceleration, $\bar{\delta}_\tau^{\mathrm{QP}} \geq \bar{\delta}_\tau(a, F_h)$ over the acceleration box.

Problem statement. Given a nominal controller supplying a^0, find a correction Δa such that:

1. $a = a^0 + \Delta a$ stays close to the nominal request;
2. the realized torque stays inside the actuator box over the horizon, allowing for uncertainty;
3. the predicted state satisfies running workspace and speed constraints; and
4. the result can be tested against sufficient conditions for transferring an independently established behavior certificate.

V. The Predictive Correction and What the Manager Reports

A. The predictive correction

The nominal controller and the map $H(x)$ are evaluated, and the final torque-box projection applied, every fast-loop period T_f; the manager — including the rate constraint on successive planned steps introduced below, which is evaluated only when the QP is solved — runs every manager period Δt, much longer than T_f. Within one manager update we write a_i for the request at horizon step $i = 0, \dots, N-1$ and a_i^0 for what the nominal controller would ask there; the manager-update index is suppressed since everything below lives inside a single update. At $i = 0$, a_{-1} denotes the manager's own solved a_0 from the *start* of the previous update, not that update's uncorrected nominal, so the rate constraint and smoothing term below bound change relative to the manager's own last decision.

The decision variable is the *complete* request a_i, not the correction — the constraints are physical conditions on the actual request, and the correction is read off afterwards:

$$
\begin{aligned}
\min_{a_0,\dots,a_{N-1}} \quad & \sum_{i=0}^{N-1} \Big(\underbrace{\|a_i - a_i^0\|_W^2}_{\text{stay close to the controller}} + \underbrace{\|a_i - a_{i-1}\|_{W_\Delta}^2}_{\text{stay smooth}} \Big) \\
\text{subject to, for each } i: \quad & p_{i+1} = p_i + \Delta t\, v_i + \tfrac{1}{2}(\Delta t)^2 a_i, \qquad v_{i+1} = v_i + \Delta t\, a_i, && \text{(prediction)} \\
& \tau_{\min} + \bar{\delta}_{\tau,i}^{\mathrm{QP}} \leq \tau_{\mathrm{base}}(\hat{x}_i) + H(\hat{x}_i)\,(a_i - G_F(\hat{x}_i)\hat{F}_{h,i}) \leq \tau_{\max} - \bar{\delta}_{\tau,i}^{\mathrm{QP}}, && \text{(realizable)} \\
& |v_i + \Delta t\, a_i| \leq v_{\max} - \epsilon_v, && \text{(velocity certificate)} \\
& z_{i+1} \in \mathcal{X}, && \text{(workspace/speed)} \\
& \|a_i\|_\infty \leq a_{\max}, \quad \|a_i - a_{i-1}\|_\infty \leq \dot{a}_{\max} \Delta t. && \text{(request-rate constraint)}
\end{aligned}
$$

Every constraint is linear in a_i and the cost is convex quadratic, so this is a small dense QP. Here $\hat{x}_i$ is the state on the fixed nominal rollout used to assemble it, $\hat{F}_{h,i}$ is the forecast interaction force at horizon step i — by default the measured force held constant across the horizon (a zero-order hold; Section VII.B's preview-mismatch case is exactly the failure mode of this choice), with other preview policies evaluated in the ablations of Section VII.E — and $\mathcal{X} = \{z : |p| \leq \mathrm{pos}_{\max},\ |v| \leq v_{\max}\}$ is the running workspace-and-speed box on the task state $z = [p; v]$. The velocity-certificate row is the set $\mathcal{K}_v$ of Section VI, written out: it replaces the ordinary predicted-speed bound $v_{\max}$ inside $\mathcal{X}$ with the tighter $v_{\max} - \epsilon_v$ for one manager step ahead. Together with the realizable-set row, a feasible solution satisfies $a_i \in \mathcal{A}^{\mathrm{tight}}(\hat{x}_i) \cap \mathcal{K}_v(v_i)$ by construction — the certified-action-set and tightened-actuator parts of Theorem 1's premise, not a check applied afterward. State, acceleration, certificate, and rate bounds are all hard; there are no slack variables and no separately constructed terminal invariant set.

The published correction is the first-step gap:

$$\Delta a = a_0 - a_0^0.$$

Interpretation. The manager first checks the nominal rollout against every constraint above; if it already satisfies all of them, the rollout is returned unchanged and $\Delta a = 0$. Otherwise the QP is solved, returning the minimum-cost adjustment — jointly penalizing deviation from the nominal behavior and variation of the corrected request over the whole horizon — that pulls the request back onto the boundary of the feasible set.

The QP is assembled *along the frozen nominal rollout*, which keeps each realizable-set constraint linear; if that rollout is wrong — from preview mismatch, or because the QP's own correction moves the trajectory away from the nominal one — the future geometry used by the current plan may become inaccurate. The present benchmark does not provide an analytical bound for this horizon-geometry drift — $\bar{\delta}_\tau$ and condition (T1) instead bound realization-map and model uncertainty evaluated at the current state (Section VII.D), a related but distinct error source. Frequent replanning limits how long any one horizon's geometry is trusted, while the model- and preview-mismatch cases expose horizon-geometry drift's consequences empirically (Section VII.B).

If the solver reports infeasibility, the implementation instead projects the first nominal acceleration onto the current tightened torque and state halfspaces (a continuous-time control-barrier-function set, not the QP's own discrete constraint) and tiles that reactive command over the stored sequence, returning zero acceleration if even that is empty; the final high-rate projection remains active either way. This fallback gives a deterministic bounded command but does **not** recover horizon feasibility or satisfy the transfer conditions — trajectories containing fallback steps are not horizon-MPC behavior.

B. What the manager reports

For each joint, the raw torque headroom still unused; the reported margin is the worst joint:

$$\mu = \min_j \left\{ \tau_{\max,j} - \tau_j,\ \tau_j - \tau_{\min,j} \right\}.$$

So $\mu > 0$ means every joint is strictly inside its limits, $\mu = 0$ means some joint is exactly at a limit, and $\mu < 0$ means the request is not realizable. The benchmark uses symmetric limits, $\tau_{\min,j} = -L_j$ and $\tau_{\max,j} = L_j$, for which this reduces to $\min_j(L_j - |\tau_j|)$. μ is reported in newton-metres, not normalized by each joint's own range — a joint with a wider torque range simply has more raw headroom to report, so μ is not comparable across robots with different actuator limits. Over the horizon we report the smallest value.

Scalar margin does not reveal whether the robot can still accelerate *in the direction it needs to*. Given a *feasible* current acceleration $a_c \in \mathcal{A}^{\text{tight}}(x)$ and a unit direction y, the remaining authority along y is how far one can move before leaving the tightened feasible set, so the indicator counts the same uncertainty margin the optimizer already enforces:

$$\alpha^+(x, a_c, y) = \max \left\{ \alpha \geq 0 :\ a_c + \alpha y \in \mathcal{A}^{\text{tight}}(x) \right\}.$$

$\mathcal{A}^{\text{tight}}(x)$ is a polytope defined by one tightened slab per actuator, $\{a : l_j \leq h_j(x)^\top a \leq u_j\}$, where $h_j(x)^\top$ is the j-th row of $H(x)$; it need not itself be an affine image of a box, but this is not required for α^+: it is still a closed-form evaluation and not a search, directly from the halfspace form $\{v : Av \leq b\}$ of $\mathcal{A}^{\text{tight}}(x)$, which any polytope in this representation admits: $\alpha^+ = \min_{j:(Ay)_j>0}(b_j - (Aa_c)_j)/(Ay)_j$, the point where the ray from a_c along y first leaves the set. If the current request is already outside $\mathcal{A}^{\text{tight}}(x)$, the manager reports $\alpha^+ = 0$ by convention rather than apply this formula, which is meaningful only from a feasible starting point. A small α^+ is **directional authority collapse**: the robot may retain plenty of unused torque overall and still be unable to resist a push along y. This is the failure a single utilization number hides.

The manager also reports whether its horizon problem is feasible. This is a useful recoverability indicator, but it is **not** membership in a certified viability kernel, because no terminal invariant set or backup policy is constructed. Section VII therefore uses the term *near-boundary braking stress case* rather than *point of no return*.

VI. Conditional Certificate Transfer

Sections IV–VI describe one robot. This section asks what survives when the robot changes.

A. The setup in words

Suppose someone has already proved something about the *behavior* — for instance, that velocity never leaves a set $\mathcal{S}_v$, provided each step's request comes from some *acceptable* set of actions rather than one fixed policy, and the one-step model error stays within a margin ϵ_v. Call that the **certificate**. It says nothing about any particular robot, and it deliberately does not pin down a single control law: different robots, or the same robot at different moments, may need to pick different members of that acceptable set, and that is not a failure of the certificate — it is why the set is the reusable object, not any one policy built on top of it.

Now put a real robot underneath. Three things can go wrong:

1. the manager's torque prediction $\hat{\tau}$ may be wrong;
2. the resulting torque may not fit in the actuator box, so clipping activates and the robot stops doing what was requested;
3. even without clipping, the real robot's one-step velocity may differ from the double-integrator prediction by more than the certificate tolerates.

Theorem 1 says: rule out all three, for whatever request the manager actually picked from the acceptable set, and the certificate carries over unchanged.

B. The certified action set, and why velocity only

Let $y = v$ be the task velocity and $F_v(y, a) = y + \Delta t\, a$ its one-step update (the velocity half of the double integrator in Section IV). Define the certified region

with the margin ϵ_v satisfying $0 < \epsilon_v < v_{\max}$,

$$\mathcal{S}_v = \{y : \|y\|_\infty \le v_{\max}\}, \qquad \mathcal{E}_v = \{d : \|d\|_\infty \le \epsilon_v\},$$

and, at every $y \in \mathcal{S}_v$, the **certified action set**

$$\mathcal{K}_v(y) = \{a : \|F_v(y, a)\|_\infty \le v_{\max} - \epsilon_v\}\,.$$

In words: $\mathcal{K}_v(y)$ is every request that keeps next step's velocity inside the certified region even after absorbing a one-step model error of size up to ϵ_v. Position is deliberately left out of the certificate — it stays a plain running constraint in $\mathcal{X}$ — because velocity is the only quantity this benchmark actually measures and audits a one-step mismatch for (Section VII.D); tightening position too would not be backed by a measured quantity.

$\mathcal{K}_v$ is never empty on $\mathcal{S}_v$ whenever $\epsilon_v / \Delta t$ is within the abstract model's admissible acceleration range: decelerating at that rate removes ϵ_v of speed within one manager period, so a legal request back into the tighter band always exists, including when several velocity components sit on the boundary at once (Appendix A gives the margin this leaves against the benchmark's own acceleration bound). This is nonemptiness of $\mathcal{K}_v(y)$ alone, in the abstract velocity model; its intersection with the robot-specific tightened torque set $\mathcal{A}^{\text{tight}}(x)$ and the rate and state constraints may still be empty, which is exactly what the reactive fallback of Section V.A exists to handle, and does happen in the experiments of Section VII.

C. The theorem

Write $\Pi(x) = y = v$ for the map from the physical state to the task velocity the certificate is stated in, and let $f^d(x, \tau, F_h)$ denote the one-step physical dynamics: the state the robot actually reaches, one manager period later, after applying torque τ under force F_h from state x.

Theorem 1 (One-step conditional velocity-certificate transfer). Suppose the physical state x lies in a verified operating region, $\Pi(x) \in \mathcal{S}_v$, and the request a satisfies $a \in \mathcal{K}_v(\Pi(x))$ — any such a, not one distinguished policy. If:

(T1) Prediction accuracy. The torque that would be sent is within the stated bound of the manager's prediction,

$$|\tau^{\text{pre}} - \hat{\tau}| \le \bar{\delta}_\tau.$$

(T2) Actuator-margin test. The prediction, widened by that bound, still fits inside the actuator box,

$$\tau_{\min} + \bar{\delta}_\tau \;\le\; \hat{\tau} \;\le\; \tau_{\max} - \bar{\delta}_\tau.$$

(T3) Certificate-margin test. The one-step velocity mismatch between the real robot and the double-integrator prediction fits inside the certificate margin,

$$\left\|\Pi\big(f^d(x, \tau^{\text{pre}}, F_h)\big) - F_v(\Pi(x), a)\right\|_\infty \;\le\; \epsilon_v.$$

then clipping does not activate during the considered transition, $\tau^{\text{app}} = \tau^{\text{pre}}$, and the physical successor stays in the certified region,

$$\Pi\big(f^d(x, \tau^{\text{app}}, F_h)\big) \in \mathcal{S}_v.$$

Proof. (T1)+(T2) give $\tau_{\min} \le \tau^{\mathrm{pre}} \le \tau_{\max}$, so the clip is inactive and $\tau^{\mathrm{app}} = \tau^{\mathrm{pre}}$. (T3) places the real one-step successor inside $F_v(\Pi(x), a) \oplus \mathcal{E}_v$. This set lies in $\mathcal{S}_v$ by the definition of $\mathcal{K}_v$. □

This is a one-step result, re-checked at each update rather than a recursive-feasibility or terminal-invariance guarantee, and it certifies the manager's own frozen-instant decision rather than the drifting two-rate signal the fast loop actually applies (Section II).

D. What actually transfers

The theorem does not make feasibility universal. For each new robot one must still supply Π, $\hat{\tau}$, the bound $\bar{\delta}_\tau$, the mismatch bound, and the operating region itself. What transfers unchanged is the certificate itself — the behavior model, the certified action set $\mathcal{K}_v$, the set $\mathcal{S}_v$, and the margin ϵ_v.

Condition (T3) can be decomposed into individual error sources, offering a prospective route to an analytical bound rather than the aggregate empirical comparison the experiments actually use (Section VII):

$$\underbrace{\bar{\eta}_{\mathrm{disc}}}_{\text{discretization}} + \underbrace{\bar{\eta}_{\mathrm{hold}}}_{\text{zero-order hold}} + \underbrace{\bar{\eta}_{\mathrm{sec}}}_{\text{secondary channels}} + \underbrace{L_F \bar{\delta}_F}_{\text{force-model error}} + \underbrace{L_\tau \bar{\delta}_\tau}_{\text{torque error}} \le \epsilon_v,$$

where L_F and L_τ convert a force-model error and a torque error into the resulting one-step velocity error.

VII. Experiments

A. Protocol

The benchmark is deterministic and uses a two-dimensional interaction task. The nominal controller and final projection run at $1\ \mathrm{kHz}$; the manager uses a $50\ \mathrm{Hz}$ predictive rate and horizon $N = 12$, with the $1\ \mathrm{kHz}$ final projection providing inter-update actuator protection; adequacy of the slower rate is operating-region and model dependent. Three configuration-dependent realization maps are evaluated: a planar 2R map, an FR3-inspired surrogate, and a six-axis-arm surrogate. The latter two reproduce different actuator geometries and limits but are not manufacturer-accurate rigid-body models.

The behavior model, predictive objective, and audit threshold $\epsilon_{\mathrm{audit}} = 0.03\ \mathrm{m/s}$ are held fixed throughout, equal to the design radius ϵ_v of Section VI. The simulation constructs and enforces the velocity certificate $(\mathcal{S}_v, \mathcal{K}_v, \epsilon_v)$; it does not construct a position-aware certificate. Future actuator-limit scales are supplied exactly from each scenario's schedule throughout the benchmark, not only in the horizon-ramp case of Fig. 1; the manager therefore anticipates a known or forecast limit schedule rather than discovering an unannounced derating. Interaction-force preview is different: unless an oracle ablation is selected, the measured force is held constant over the horizon (Section V.A).

The five nominal-controller interfaces are PD, impedance, a small trained policy, a fitted neural policy, and a conditioned motion primitive (training and fitting details in Appendix A). The last two test the command/preview interface only; they are not evidence of semantic AI safety or of improved policy quality.

The 111 cases comprise 40 scenario cases, 30 controller-interface cases, 24 cross-realization cases, and 17 ablations grouped into eight families (full accounting in Appendix A).

Reported quantities are pre-clip torque excess, applied torque excess, workspace excess, behavior-realization RMSE, warning lead time, directional authority, observed one-step mismatch, and computation time. For a two-dimensional residual r_k, RMSE is the pooled component-wise value $\sqrt{\frac{1}{2K}\sum_k \|r_k\|_2^2}$. The mismatch in Table III is instead a maximum ℓ_∞ norm, matching Theorem 1's (T3), and is not directly comparable with it. Applied torque remains inside its box whenever the final projection is active.

B. Anticipatory saturation management

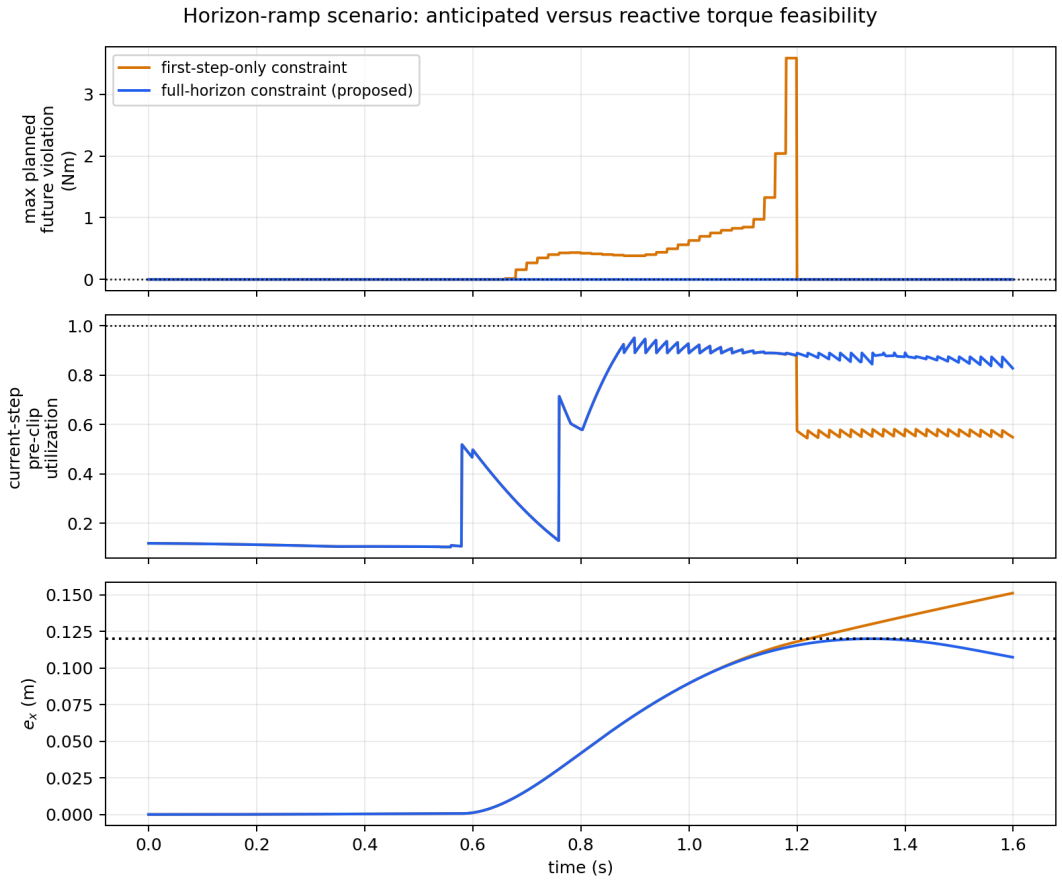

Fig. 1. Horizon-ramp scenario: a first-step-only constraint sees a legal present and walks into an illegal future; the full-horizon constraint sees it coming.

Fig. 1 isolates why anticipation is a different constraint from reactive limiting, not a refinement of it:

- The manager's horizon rows query the scenario's scheduled actuator-budget drop directly, so Fig. 1 demonstrates the horizon-versus-one-step mechanism under a known future constraint, not forecasting of an unknown one.
- The first-step-only constraint's predicted future torque violation reaches $3.587\ \mathrm{Nm}$; the full-horizon constraint stays at zero throughout.
- The first-step-only variant's QP is feasible for only 75% of updates, versus 100% for the full-horizon variant.
- The resulting $31.180\ \mathrm{mm}$ of workspace excess occurs entirely during the infeasible 25%, mediated by the reactive fallback of Section V.A ($0.000\ \mathrm{mm}$ while the QP is feasible).

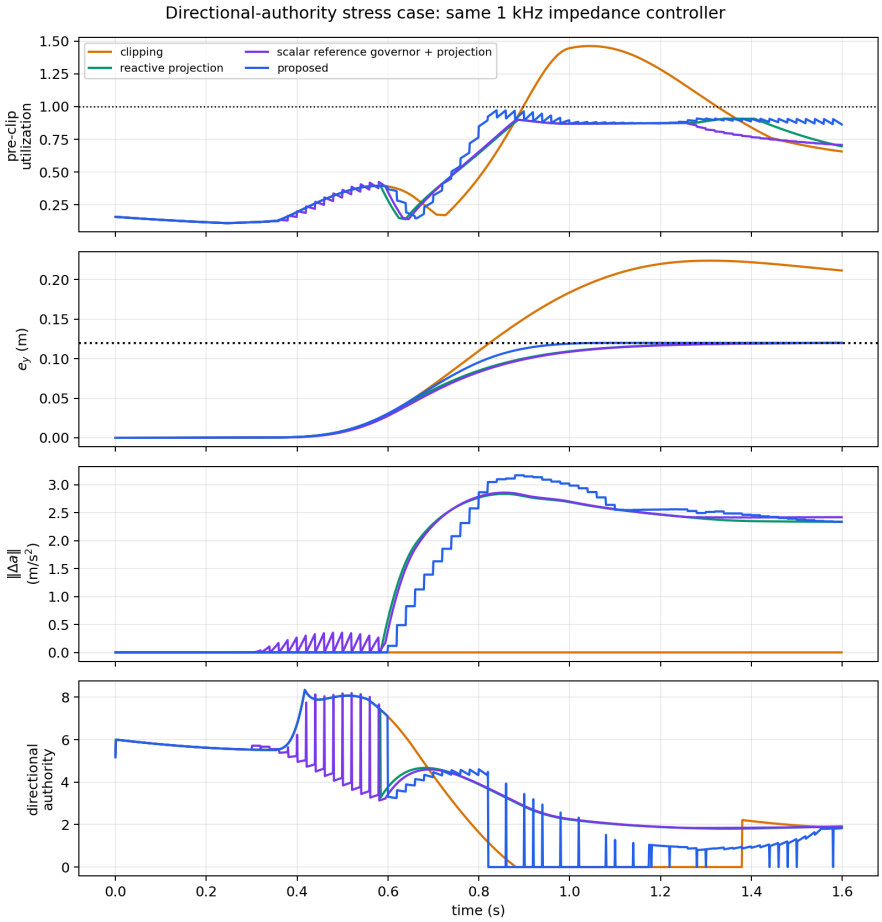

Fig. 2. Directional-authority stress case with a common impedance-controller interface.

Fig. 2 shows the directional-authority stress case. Direct clipping exceeds both the feasible request set and the workspace bound. The predictive methods intervene earlier, and the vector correction preserves the workspace constraint while leaving a visible intervention residual. Measured along the disturbance direction (metric defined in Appendix A), the proposed manager's authority reaches zero for about a fifth of the trajectory while the applied request keeps the workspace constraint satisfied throughout; the reactive projection and scalar governor retain positive authority in this direction across the whole run (minimum 1.82 and $1.87\ \mathrm{m/s^2}$).

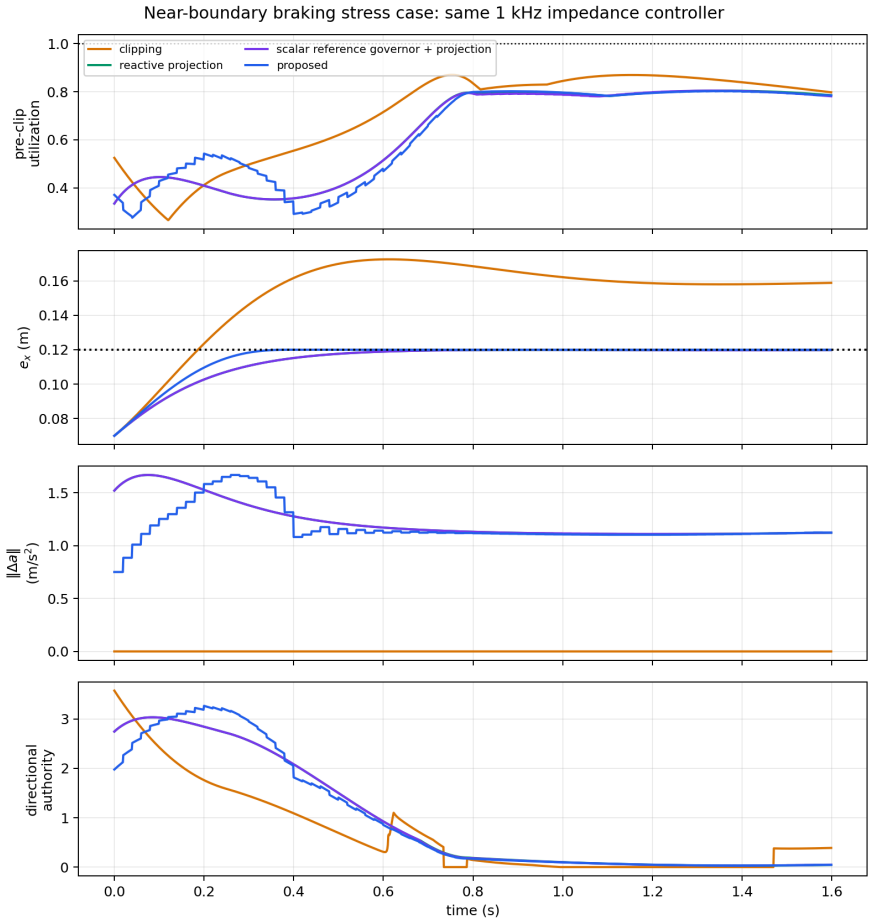

Fig. 3. Near-boundary braking stress case; no viability kernel is inferred from this trajectory.

Fig. 3 shows near-boundary braking: an outward velocity close to the position boundary under a shrinking torque budget. Direct clipping overshoots the boundary and settles outside it. The reactive projection, scalar reference governor plus projection, and proposed manager all arrest the position at the boundary. Because no viability kernel or terminal invariant set is computed, this supports finite-horizon constraint handling only.

Table I reports clipping and proposed results across the remaining seven scenarios. Unlike the horizon-ramp scenario of Fig. 1, none of these is built to isolate the anticipation mechanism specifically, which is why the lead-time differences below are modest — that isolated demonstration is the job of Fig. 1, not of this table.

The manager, reactive projection, and scalar governor all warn before the limiting event across the sampled cases (Table I), with leads differing across methods by at most a few tens of milliseconds, comparable to the shared $20\ \mathrm{ms}$ manager-update interval; the prediction horizon is $N\Delta t = 0.24\ \mathrm{s}$. The same relative ordering holds whether lead is measured against each method's own trigger or a shared reference event. This is a descriptive comparison against the implemented *scalar* governor only, not evidence of superiority over directional or vector reference governors.

Scenario	Pre-clip C/P (Nm)	Workspace C/P (mm)	Lead (s)	QP feasible	Audit
No saturation	0.000 / 0.000	0.000 / 0.000	–	100%	Yes
Slow saturation	0.000 / 0.000	78.970 / 0.001	0.412	100%	Yes
Sudden disturbance	7.208 / 11.976	0.000 / 0.000	0.084	92.5%	No
Directional collapse	0.693 / 0.000	103.914 / 0.003	0.339	100%	Yes
Near-boundary braking	0.000 / 0.000	52.537 / 0.011	0.566	100%	Yes
Model mismatch	12.665 / 1.981	190.434 / 193.598	0.558	45.0%	No
Preview mismatch	19.917 / 4.409	190.191 / 344.116	0.170	40.0%	No

Table I. Scenario results. C/P denotes clipping/proposed.

Negative cases. QP feasibility is 100% in the four successful stress cases but only 92.5%, 45.0%, and 40.0% under sudden disturbance, model mismatch, and preview mismatch, with the reactive fallback of Section V.A carrying most of the resulting excess. Under preview mismatch, the correction acts on a force forecast that misses a sign change inside the horizon and workspace excess rises from 190.191 to $344.116\ \mathrm{mm}$ — substantially worse than doing nothing; model mismatch is similarly worse, 193.598 versus $190.434\ \mathrm{mm}$, because the feasible geometry is assembled along a wrong rollout. Under sudden disturbance the implemented preview holds the measured force constant, so an unannounced impulse pushes pre-clip excess from 7.208 to $11.976\ \mathrm{Nm}$ even though the final projection keeps applied torque legal. The audit rejects all three cases, flagging exactly where Theorem 1's premises are not met.

C. Controller-interface substitution

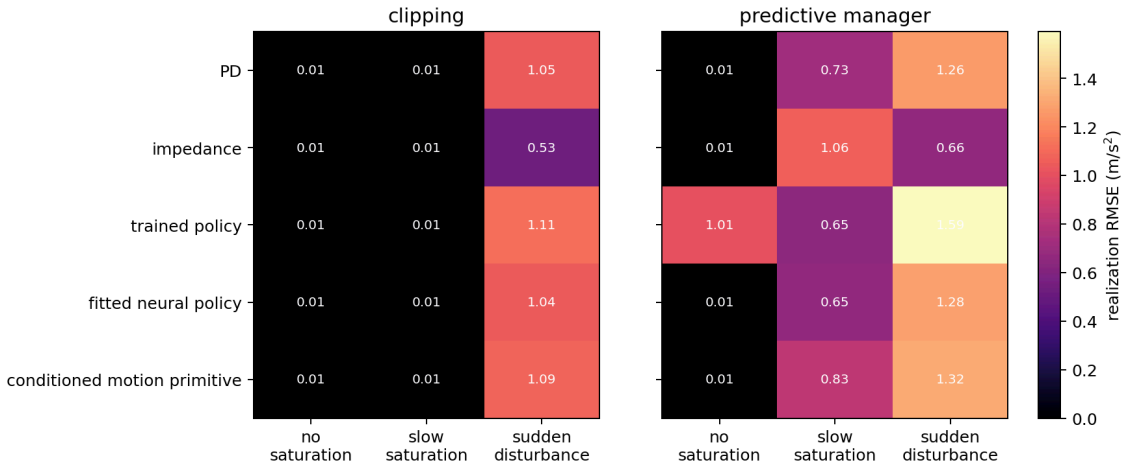

Fig. 4. Behavior-realization residuals for the five nominal-controller interfaces.

As shown in Fig. 4, the manager formulation and weights are unchanged across interfaces. Under no saturation, correction RMSE is below $0.01\ \mathrm{m/s^2}$ for four of five controllers; the small evolution-strategy policy is the exception at $\approx 1.01\ \mathrm{m/s^2}$ (bias source in Appendix A). The manager holds the state inside the workspace box but is compensating for a biased interface rather than remaining inactive. The case is kept as an interface stress test — the architecture is not intended to repair behavior-policy quality, though its running constraints may incidentally reject a biased request.

Table II separates the manager's deliberate correction ($a_{\mathrm{req}} - a^0$) from the tracking-defect RMSE ($a_{\mathrm{actual}} - a_{\mathrm{req}}$) most directly tied to certificate transfer. The tracking defect is $0.0111\ \mathrm{m/s^2}$ for every interface — a fixed, controller-independent unmodeled-error floor, since no clipping is active under slow saturation — while correction RMSE varies with what each controller requests. All five cases pass the audit.

Interface	Correction RMSE ($\mathrm{m/s^2}$)	Tracking defect RMSE ($\mathrm{m/s^2}$)	Workspace excess (mm)	Audit
PD	0.726	0.0111	0.001	Pass
Impedance	1.060	0.0111	0.001	Pass
Trained policy	0.652	0.0111	0.016	Pass
Fitted neural policy	0.649	0.0111	0.001	Pass
Conditioned motion primitive	0.830	0.0111	0.006	Pass

Table II. Controller substitution under slow saturation.

D. Sampled interface audit across realization maps

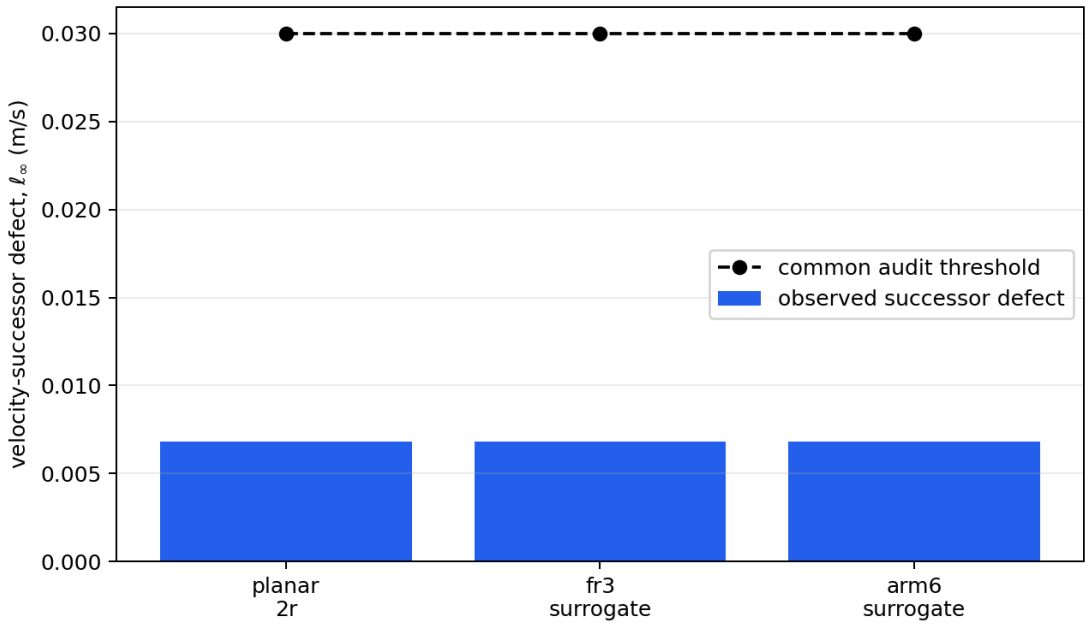

Fig. 5. Observed one-step mismatches versus the common audit threshold.

Realization map	Max. T3 defect, ℓ_∞ (m/s)	Min. T1 slack (Nm)	Min. T2 slack (Nm)
Planar 2R	0.006815	0.002637	0.143756
FR3-inspired surrogate	0.006796	2.36×10^{-5}	0.646664
Six-axis-arm surrogate	0.006796	1.27×10^{-4}	0.707748

Table III. Sampled start/end quantities associated with the three conditions of Theorem 1.

Each record is indexed by one feasible manager update, excluding ticks following an infeasible (fallback) solve since Theorem 1 concerns the QP-optimized plan. (T1) and (T2) are evaluated at the beginning of the interval, using the manager's own first-horizon-step prediction $\hat{\tau}_0$ and the torque realized from that same request. Both use the same state, force, and request, though they are not numerically equal because their gap is exactly what (T1) bounds. The (T2) entry is that tick's first-step slack, not the horizon-wide minimum reported elsewhere. (T3) uses the physical state observed at the *next* manager update. The table therefore reports a sampled start/end audit — 316 records per robot from the no-saturation and slow-saturation cross-realization runs — rather than asserting that its start-of-interval (T1) and (T2) values hold continuously while the fast controller and realization map evolve over the intervening $20\ \mathrm{ms}$.

The pass/fail label in Tables I and II comes from a separate, broader unpaired audit over every $1\ \mathrm{kHz}$ fast-loop sample. It uses a Euclidean (T3) bound, conservative relative to Table III's ℓ_∞ value, checks the torque-error envelope, and directly requires realized pre-clip torque to remain inside the untightened actuator box. Thus the statement that pre-clip torque remains legal throughout each passing trajectory is supported by the fast-loop check, not inferred from Table III's manager-update snapshots alone. Fig. 5 and Table III show positive sampled slacks across all three realization maps: the (T3) defect remains below the $0.03\ \mathrm{m/s}$ certificate margin, while (T1) and (T2) retain room against $\bar{\delta}_\tau(a, F_h)$ (Appendix A). Because the injected errors are drawn from the same envelope against which they are checked, this is a consistency check on the mechanism rather than independent uncertainty validation.

E. Ablation summary and computation

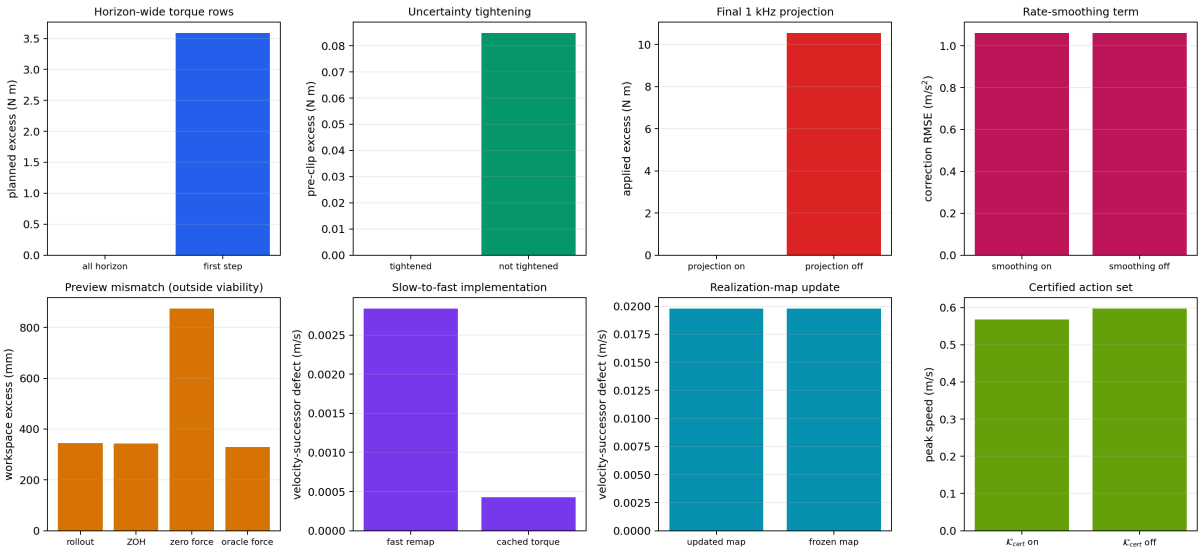

Fig. 6. Paired ablations of the predictive and fast-path implementation choices.

Fig. 6 isolates the four design choices central to the paper. Full-horizon enforcement keeps the horizon-ramp QP feasible and removes the $3.587\ \mathrm{Nm}$ planned violation left by first-step-only enforcement. Removing uncertainty tightening creates $0.0848\ \mathrm{Nm}$ of pre-clip excess; removing the final projection creates $10.538\ \mathrm{Nm}$ of applied excess under sudden disturbance. In the dedicated velocity-bound case, enforcing $\mathcal{K}_v$ limits peak speed to $0.5678\ \mathrm{m/s}$, versus $0.5968\ \mathrm{m/s}$ without it. Appendix A gives the complete 17-run ablation ledger, secondary results, and wall-clock computation figures.

VIII. Discussion

The experiments support a narrower conclusion than “one safety controller works for every robot.” The predictive optimization statement and the empirical audit threshold remain unchanged across the implemented controller interfaces and realization maps. Physical feasibility does not transfer automatically: each robot must reconstruct and verify its feasible set and its error bounds.

This also clarifies the relation to a reference governor or predictive safety filter. Those architectures can adopt the same behavior coordinates. What the separation adds is an explicit proof boundary — a reusable behavior certificate on one side, a checkable robot-specific realization contract on the other. In the present study the velocity certificate $(\mathcal{S}_v, \mathcal{K}_v, \epsilon_v)$ is concretely constructed and enforced, but the physical containment it also requires — that the real one-step mismatch actually stays inside ϵ_v — is sampled along experiment trajectories rather than independently identified over the workspace. If torque uncertainty exceeds the available margin, or an observed mismatch exceeds ϵ_{audit}, the audit rejects the case.

The final high-rate projection is essential but should not be confused with behavior preservation: clipping protects the actuator but invalidates behavior preservation, and Theorem 1 states when clipping remains inactive for a given transition — whenever (T1)–(T3) are re-established at the corresponding update — so the requested closed-loop behavior is actually realized. The mismatch experiments show these two properties diverging.

IX. Conclusion

This paper introduced a predictive realization architecture for actuator-limited robot controllers. The nominal controller stays at its own high servo rate; a much slower MPC layer forecasts robot-specific loss of realizability and modifies the requested acceleration before clipping is required. The separation does not make actuator feasibility universal. It identifies which object may be reused — a certified action set stated in behavior coordinates — and which must be verified again — the realization map, the actuator margin, the uncertainty bounds, and the operating region. A minimal, deliberately simple instance of that certified action set is enforced inside the QP itself, so the theorem’s key hypothesis holds by construction rather than by a check applied afterward; it is an existence witness for that kind of enforcement, not the definitive form the certificate should take.

Across 111 deterministic reduced-order runs (including repeated anchor cases across matrices), full-horizon enforcement eliminates a planned future violation that remains in the unconstrained future rollout of the first-step-only variant — the post-hoc rollout detects the violation, the constraint itself does not — and the architecture accepts five nominal-controller interfaces. A sampled interface audit applies the same mismatch threshold across three realization maps. Slow saturation, directional authority collapse, and near-boundary braking violations are prevented within the tested region. Abrupt disturbance and severe mismatch expose cases where the audit fails, the QP frequently becomes infeasible, and only the reactive fallback plus final projection remain. A dedicated ablation shows the certified action set changes the manager’s output when it binds, while leaving the eight main stress scenarios unaffected — consistent with the certificate margin going largely unused in those cases.

Future work will replace the surrogate maps with full rigid-body systems, extend the certificate from one-step velocity to position and recursive feasibility, sweep operating conditions rather than reporting single deterministic trajectories, and validate the architecture on a real-time hardware loop.

Appendix A: Benchmark Parameters

This appendix lists the constants needed to reproduce the numbers in Section VII, including the complete ablation ledger; the full realization-map functions $H(x)$, $\tau_{\text{base}}(x)$, the learned-policy weights, and the per-scenario timelines are in the supplementary simulation code, not reproduced here.

Shared constants (`BenchmarkConfig` , `simulation/saturation_benchmark.py`): fast period $T_f = 1\ \text{ms}$, manager period $\Delta t = 20\ \text{ms}$, horizon $N = 12$, episode duration $1.6\ \text{s}$, workspace box $\text{pos}_{\max} = 0.12\ \text{m}$, speed box $v_{\max} = 0.60\ \text{m/s}$, acceleration bound $a_{\max} = 12.0\ \text{m/s}^2$, acceleration rate bound $\dot{a}_{\max} = 120.0\ \text{m/s}^2\text{/s}$, certificate/audit margin $\epsilon_v = \epsilon_{\text{audit}} = 0.03\ \text{m/s}$, cost weights $W = 1.0 \cdot I$, $W_\Delta = 0.025 \cdot I$, OSQP tolerance 10^{-5}. These give $\epsilon_v / \Delta t = 1.5\ \text{m/s}^2$, well under $a_{\max}$ — the concrete margin behind $\mathcal{K}_v$’s nonemptiness (Section VI.B).

Realization maps (torque limits in Nm, mass in kg): every listed torque-limit array is the positive magnitude L of a symmetric box, $\tau_{\min} = -L$ and $\tau_{\max} = L$. Planar 2R — 2 joints, $L = [9.0, 7.5]$, mass 2.2; FR3-inspired surrogate — 7 joints, $L = [16.0, 13.0, 12.0, 15.0, 6.0, 6.0, 4.5]$, mass 3.0; six-axis-arm surrogate — 6 joints, $L = [12.0, 10.0, 9.0, 8.5, 5.5, 4.5]$, mass 2.6. The interaction-error bound $\bar{\delta}_\tau$ uses base term $0.03\ \mathrm{Nm}$ per joint and the H-dependent term below, with $H_0 = H(x)$ evaluated once at each map's zero-state configuration.

Cross-realization audit detail. The per-joint bound is

$$\bar{\delta}_\tau^{(j)} = s_{\mathrm{mm}} \left(\underbrace{0.03}_{\text{Nm, base-torque term}} + 0.008 \sum_k \max\Big(|H_0|_{jk}\,,\, 0.25\ \mathrm{Nm\,s^2/m} \Big) \left| a_k - \frac{F_{h,k}}{m} \right| \right),$$

where s_{mm} is the scenario's mismatch-scale multiplier (1.0 for the cross-realization cases reported here, up to 2.4 for model mismatch elsewhere in Section VII), ranging from 0.0300 to $0.0926\ \mathrm{Nm}$ over the audit; this is $\bar{\delta}_\tau(a, F_h)$, the action-dependent bound of Section IV, not the QP's own more conservative $\bar{\delta}_\tau^{\mathrm{QP}}$. Table III reports the resulting (T1) and (T2) slacks directly.

Baseline architectures. Reactive $1\ \mathrm{kHz}$ projection: exact Euclidean projection onto the intersection of the current-step tightened torque halfspaces and a relative-degree-two box control-barrier-function halfspace set with decay rate $\lambda = 8$ (`reactive_state_halfspaces`), i.e. position rows $\mp a \le \pm 2\lambda v + \lambda^2(\mathrm{pos}_{\max} \mp p)$ and speed rows $\mp a \le \lambda(v_{\max} \mp v)$. Scalar reference governor: at each manager tick, finds the largest scalar $\alpha \in [0, 1]$ such that following α of the way from the current state toward the goal keeps the resulting request inside the same tightened torque and state halfspaces (`governor_scale`), then hands the α-scaled request to the same reactive projection above — so the governor differs from plain reactive projection only in using a single scalar authority over the whole request rather than an independent per-direction one.

Directional-authority metric. α^+ (Section V.B) is a function of a chosen direction y. Fig. 2 reports it along the fixed, exogenous disturbance direction rather than the manager's own correction direction, which is endogenous — it can change discontinuously as the manager's own output changes — and so is not directly comparable across methods or time; both use the same halfspace-ray construction.

Governor-search validation. The scalar-governor implementation uses bisection over α, which assumes feasibility is monotone in α. A dense 201-point grid check across all 160 manager ticks at which the governor is invoked in this benchmark found no non-monotone case.

Controller interfaces. PD and impedance are closed-form analytic laws. The trained policy is a small $\tanh$-squashed single-hidden-layer network fit by a deterministic evolution strategy. The fitted neural policy is an 18-unit $\tanh$ hidden layer with a linear output head, least-squares fit to impedance-law demonstrations (`saturation_benchmark.py` , `RLPolicyController` /neural-controller fit). The conditioned motion primitive is a PD servo tracking a primitive-conditioned reference.

Trained-policy bias (Section VII.C). The evolution-strategy policy's limited training set does not cover the test trajectory symmetrically, and it retains a positive-y command bias; this is an observed generalization error, not a specific failure of the evolution-strategy algorithm.

Computation. In the saved run, both the manager's and the fast path's worst observed wall-clock maxima remained below their nominal periods (20 ms and 1 ms respectively), with median solve times well under either budget. These are wall-clock measurements of computational cost on a development machine, not a real-time guarantee; because they are nondeterministic run-to-run, the released `results/all_experiment_metrics.json` is the source of record rather than any specific figure quoted here. The simulator itself advances the physical state by exactly $1\ \mathrm{ms}$ every step and does not model a scheduler that drops or delays steps on a missed deadline. A hard real-time implementation with an explicit scheduling policy is future work.

Case accounting. The 111 cases comprise 40 scenario cases, 30 controller-interface cases, 24 cross-realization cases, and 17 ablations grouped into eight families. The 40 scenario cases are eight scenarios evaluated with five channels: four architectures — direct clipping, a reactive $1\ \mathrm{kHz}$ projection, a scalar reference governor followed by the same reactive projection, and the proposed horizon-wide correction with final projection — plus one unconstrained nominal-reference channel without torque projection. The stress scenarios are no saturation, slow saturation, sudden disturbance, directional authority collapse, near-boundary braking, model mismatch, preview mismatch, and a horizon-ramp scenario reported directly as Fig. 1 and used for the constraint-width ablation of Section VII.E. A ninth scenario, starting inside $\mathcal{S}_v$ near its tightened boundary, is used only for the velocity-certificate ablation and enters neither the scenario nor the cross-realization matrices.

Complete ablation ledger. All ablations use the FR3-inspired surrogate and impedance controller.

#	Scenario	Variant	Changed element
1	Horizon ramp	Full	Complete proposed manager
2	Horizon ramp	First-step torque	Torque constraints only at the first horizon step
3	Slow saturation	Cached full torque	Hold the complete torque command between manager updates
4	Sudden disturbance	Full	Complete proposed manager
5	Sudden disturbance	No final projection	Disable the $1\ \mathrm{kHz}$ torque projection
6	Model mismatch	Full	Complete proposed manager
7	Directional collapse	Full	Complete proposed manager
8	Directional collapse	No tightening	Set $\bar{\delta}_\tau = 0$ in the predictive constraints
9	Model mismatch	Frozen realization map	Freeze H and τ_{base} at the current manager state
10	Model mismatch	Updated realization map	Evaluate the maps along the nominal predicted states
11	Preview mismatch	Full	Measured-force hold preview
12	Preview mismatch	Acceleration ZOH	Hold the current nominal acceleration over the horizon
13	Preview mismatch	Zero-force preview	Set the forecast force to zero
14	Preview mismatch	Oracle-force preview	Supply the scenario's future force
15	Slow saturation	No smoothing	Set $W_\Delta = 0$
16	Certificate-margin case	Constrained	Enforce $\mathcal{K}_v$
17	Certificate-margin case	Unconstrained	Remove $\mathcal{K}_v$

Secondary ablation results. Removing request smoothing leaves correction RMSE under slow saturation essentially unchanged ($1.0603\ \mathrm{m/s^2}$ without it versus $1.0604\ \mathrm{m/s^2}$ with it). Zero-force preview increases workspace excess from 344.116 to $873.921\ \mathrm{mm}$, but no preview option restores viability in the severe mismatch case. Recomputing the fast map does not outperform cached torque in this

reduced-order benchmark, and updating it is numerically indistinguishable from freezing it. These are reported as non-results rather than evidence for those mechanisms.

Reproducibility. The supplementary archive accompanying this anonymous submission contains the complete code, tests, saved metrics, and figure-generation scripts. The complete configuration — including the ablation-family definitions and scenario force/goal timelines — is fixed in `simulation/saturation_benchmark.py` and `simulation/run_all_experiments.py`. The evolution-strategy policy uses RNG seed 4, and the fitted neural policy uses seed 7. From the archive's `pHRI/saturation` directory, the complete deterministic suite is regenerated with:

```
python3 -m pip install -r requirements.txt
MPLCONFIGDIR=/tmp/mpl-saturation \
XDG_CACHE_HOME=/tmp/cache-saturation \
PYTHONPATH=simulation \
python3 simulation/run_all_experiments.py
```

The command writes `results/all_experiment_metrics.json` and the figures used in Section VII.

References

[1] N. Hogan, "Impedance Control: An Approach to Manipulation—Part I: Theory," *Journal of Dynamic Systems, Measurement, and Control*, vol. 107, no. 1, pp. 1–7, 1985, doi: 10.1115/1.3140702.

[2] O. Khatib, "A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation," *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987, doi: 10.1109/JRA.1987.1087068.

[3] E. Garone, S. Di Cairano, and I. Kolmanovsky, "Reference and Command Governors for Systems with Constraints: A Survey on Theory and Applications," *Automatica*, vol. 75, pp. 306–328, 2017, doi: 10.1016/j.automatica.2016.08.013.

[4] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *Proceedings of the European Control Conference*, pp. 3420–3431, 2019, doi: 10.23919/ECC.2019.8796030.

[5] K. P. Wabersich and M. N. Zeilinger, "Linear Model Predictive Safety Certification for Learning-Based Control," in *Proceedings of the IEEE Conference on Decision and Control*, 2018.

[6] Y.-Y. Cao, Z. Lin, and D. G. Ward, "An Anti-Windup Approach to Enlarging Domain of Attraction for Linear Systems Subject to Actuator Saturation," *IEEE Transactions on Automatic Control*, vol. 47, no. 1, pp. 140–145, 2002, doi: 10.1109/9.981734.

[7] Y.-Y. Cao, Z. Lin, and D. G. Ward, "Anti-Windup Design of Output Tracking Systems Subject to Actuator Saturation and Constant Disturbances," *Automatica*, vol. 40, no. 7, pp. 1221–1228, 2004, doi: 10.1016/j.automatica.2004.02.012.

[8] A. Girard and G. J. Pappas, "Approximation Metrics for Discrete and Continuous Systems," *IEEE Transactions on Automatic Control*, vol. 52, no. 5, pp. 782–798, 2007.

[9] A. Girard and G. J. Pappas, "Hierarchical Control System Design Using Approximate Simulation," *Automatica*, vol. 45, no. 2, pp. 566–571, 2009.

[10] P. Nuzzo, J. B. Finn, A. Iannopollo, and A. Sangiovanni-Vincentelli, "Contract-Based Design of Control Protocols for Safety-Critical Cyber-Physical Systems," in *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition*, 2014.

[11] M. Sharifi, H. Salarieh, S. Behzadipour, and M. Tavakoli, "Nonlinear Model Reference Adaptive Impedance Control for Human–Robot Interactions," *Control Engineering Practice*, vol. 32, pp. 9–27, 2014, doi: 10.1016/j.conengprac.2014.07.001.

[12] L. Roveda, A. Testa, A. A. Shahid, F. Braghin, and D. Piga, "Q-Learning-Based Model Predictive Variable Impedance Control for Physical Human–Robot Collaboration," *Artificial Intelligence*, vol. 312, art. 103771, 2022, doi: 10.1016/j.artint.2022.103771.

[13] K. Haninger, C. Hegeler, and L. Peternel, "Model Predictive Impedance Control with Gaussian Processes for Human and Environment Interaction," *Robotics and Autonomous Systems*, vol. 165, art. 104431, 2023, doi: 10.1016/j.robot.2023.104431.

[14] A. S. Anand, J. T. Gravdahl, and F. J. Abu-Dakka, "Model-Based Variable Impedance Learning Control for Robotic Manipulation," *Robotics and Autonomous Systems*, vol. 170, art. 104531, 2023, doi: 10.1016/j.robot.2023.104531.

[15] J. Xue, W. Liang, Y. Wu, and T. H. Lee, "Model Predictive Variable Impedance Control Towards Safe Robotic Interaction in Unknown Disturbance-Rich Environments," *Robotics and Autonomous Systems*, vol. 189, art. 104961, 2025, doi: 10.1016/j.robot.2025.104961.

[16] K. P. Wabersich and M. N. Zeilinger, "A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems," *Automatica*, vol. 129, art. 109597, 2021, doi: 10.1016/j.automatica.2021.109597.